Отчет первого блока

ФИО: Михайлов Петр Сергеевич
ИСУ: 466730
Группа: P3211

1 Агроном - любитель

1.1 Код задачи

```cpp
#include <iostream>
#include <vector>

using namespace std;

int main() {
  int n;
  if (!(cin >> n)) {
    return 0;
  }
  vector<int> a(n + 1);
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
  }
  int max_len = 0;
  int best_l = 1;
  int best_r = 1;

  int l = 1;

  for (int r = 1; r <= n; r++) {
    if (a[r] == a[r - 1] && a[r] == a[r - 2]) {
      l = r - 1;
    }
    int current_len = r - l + 1;
    if (current_len > max_len) {
      max_len = current_len;
      best_l = l;
      best_r = r;
    }
  }
  cout << best_l << " " << best_r << endl;

  return 0;
}
```

1.2 Этапы мышления

Основная идея: Поиск самого длинного непрерывного подлинного отрезка, в котором ни один сорт цветов не встречается более двух раз подряд.

Алгоритм:

- Используется метод двух указателей: `l` (левая граница текущего отрезка) и `r` (правая граница).
- В цикле проходим по всем элементам массива. Как только обнаруживается «тройка» одинаковых элементов подряд ($a[r] = a[r-1] = a[r-2]$), левая граница `l` перемещается на позицию $r - 1$. Это позволяет исключить первый элемент тройки, сохранив максимально возможную длину текущего окна.
- На каждой итерации вычисляется текущая длина и, если она больше максимальной, обновляются координаты `best_l` и `best_r`.

2 Зоопарк Глеба

2.1 Код задачи

```cpp
#include <cctype>
#include <iostream>
#include <stack>
#include <string>
#include <vector>

using namespace std;

struct Object {
  char type;  // A, a, B, b ...
  int id;     // Нмоер во входной строке
};

int main() {
  string s;
  if (!(cin >> s)) {
    return 0;
  }

  int n = s.length() / 2;  // Кол-во видов
  stack<Object> st;
  vector<int> result(n + 1);

  int animals = 0;
  int traps = 0;
  vector<int> current_id(s.length());
  for (int i = 0; i < static_cast<int>(s.length()); i++) {
    if (islower(s[i]) != 0) {
      current_id[i] = ++animals;
    } else {
      current_id[i] = ++traps;
    }
  }
  for (int i = 0; i < static_cast<int>(s.length()); i++) {
    if (!st.empty()) {
      char top_char = st.top().type;
      char current_char = s[i];

      if (tolower(top_char) == tolower(current_char) && top_char != current_char) {
        int top_index = current_id[st.top().id];
        int curr_index = current_id[i];
        if (islower(current_char) != 0) {  // curr - животное, top - ловушка
          result[top_index] = curr_index;
        } else {  // наоборот
          result[curr_index] = top_index;
        }

        st.pop();
        continue;
      }
    }
    st.push({s[i], i});
  }

  if (st.empty()) {
    cout << "Possible" << endl;
    for (int i = 1; i <= n; i++) {
      cout << result[i] << (i == n ? "" : " ");
    }
    cout << endl;
  } else {
    cout << "Impossible" << endl;
  }

  return 0;
}
```

2.2 Этапы мышления

Основная идея: Использование структуры данных «стек» для сопоставления животных и ловушек по принципу правильной скобочной последовательности.

Алгоритм:

- **Предварительная индексация:** Сначала проходим по строке и присваиваем каждому животному (строчные буквы) и каждой ловушке (заглавные буквы) свой порядковый номер среди своего вида.
- **Сопоставление через стек:** Проходим по строке второй раз. Если текущий символ и символ на вершине стека — это одна и та же буква разного регистра (пара «животное-ловушка»), мы извлекаем элемент из стека и записываем результат сопоставления в массив `result`. В противном случае — кладем текущий символ в стек.
- **Результат:** Если в конце стек пуст, значит все пары найдены. Выводим массив `result`, где индексом является номер ловушки, а значением — номер соответствующего ей животного.

3 Конфигурационный файл

3.1 Код задачи

```
#include <iostream>
#include <map>
#include <string>
#include <vector>

using namespace std;

map<string, vector<int> > a;
vector<vector<string> > b;

int main() {
  string s;
  b.push_back(vector<string>());
  while (cin >> s) {
    if (s == "{") {
      b.push_back(vector<string>());
    } else {
      if (s == "}") {
        int n = (int)b.back().size();
        for (int i = 0; i < n; i++) {
          a[b.back()[i]].pop_back();
        }
        b.pop_back();
      } else {
        int p = 0;
        for (int i = 0; i < (int)s.length(); i++) {
          if (s[i] == '=') {
            p = i;
            break;
          }
        }
        string x = "";
        string y = "";
        for (int i = 0; i < p; i++) {
          x += s[i];
        }
        for (int i = p + 1; i < (int)s.length(); i++) {
          y += s[i];
        }
        int v = 0;
        if (y[0] == '-' || (y[0] >= '0' && y[0] <= '9')) {
          int c = 1;
          int st = 0;
```

```
            if (y[0] == '-') {
              c = -1;
              st = 1;
            }
            int r = 0;
            for (int i = st; i < (int)y.length(); i++) {
              r = r * 10 + (y[i] - '0');
            }
            v = r * c;
          } else {
            if (a.find(y) == a.end()) {
              v = 0;
            } else {
              if (a[y].size() == 0) {
                v = 0;
              } else {
                v = a[y].back();
              }
            }
            cout << v << "\n";
          }
          a[x].push_back(v);
          b.back().push_back(x);
        }
      }
    }
    return 0;
}
```

3.2 Этапы мышления

Основная идея: Использование словаря стеков для хранения истории значений переменных и отдельного стека блоков для отслеживания области видимости (какие именно переменные были изменены на текущем уровне вложенности).

Алгоритм:

- **Хранение значений:** Для каждой переменной заводится свой стек значений (реализованный через `vector<int>` внутри структуры `map`). Текущим (актуальным) значением переменной всегда является верхний элемент её стека. Если переменной нет или стек пуст, её значение по умолчанию равно 0.
- **Контроль блоков:** Для отслеживания изменений по уровням вложенности используется `vector<vector<string»`. Каждый внутренний вектор представляет собой блок `{}` и хранит имена переменных, которым были присвоены значения именно внутри этого блока.
- **Присваивание:** При обработке строки вида $x = y$ мы определяем значение правой части (парсим число вручную или берем актуальное значение переменной y из словаря). Полученное значение кладется на вершину стека переменной x, а само имя x записывается в текущий активный блок.
- **Откат состояния:** При закрытии блока `}` алгоритм проходит по всем именам переменных, сохраненным для данного уровня, и удаляет верхний элемент из их стеков (`pop_back()`). Таким образом происходит быстрый и корректный «откат» значений к состоянию до входа в блок.

4 Профессор Хаос

4.1 Код задачи

```cpp
#include <iostream>

using namespace std;

long long q, w, e, r, t, y;

int main() {
  if (cin >> q >> w >> e >> r >> t) {
    y = -100;
    long long u = 0;
    while (u < t) {
      long long z = q * w;
      z = z - e;
      if (z < 0) {
        z = 0;
      }
      if (z > r) {
        z = r;
      }
      if (z == y) {
        q = z;
        u = t;
      } else {
        y = z;
        q = z;
        u = u + 1;
      }
      if (q == 0) {
        u = t;
      }
    }
    cout << q << "\n";
  }
  return 0;
}
```

4.2 Этапы мышления

Основная идея: Прямое моделирование изменения численности популяции бактерий день за днем с использованием условий досрочного прерывания (стабилизации), чтобы избежать превышения лимита времени (TLE) при огромных значениях количества дней.

Алгоритм:

- В цикле последовательно вычисляется новое количество бактерий: текущая популяция умножается на коэффициент размножения (`w`), затем вычитается смертность (`e`). Полученный результат искусственно ограничивается снизу нулем, а сверху — максимальной вместимостью колбы (`r`).
- Чтобы не выполнять итерации все `t` дней, на каждом шаге алгоритм проверяет два условия для раннего выхода:
  1. **Полное вымирание:** Если количество бактерий падает до 0 (`q == 0`), популяция уже не восстановится.
  2. **Стабилизация (неподвижная точка):** Если вычисленное количество бактерий совпадает с результатом строго предыдущего дня (`z == y`), это означает, что система пришла в равновесие и значение больше меняться не будет.
- При срабатывании любого из этих условий счетчик дней переводится в конец (`u = t`), цикл прерывается, и программа мгновенно выдает ответ.